Autonomy

Poniżej znajdują się treści zadań rekrutacyjnych do podprojektu Autonomy. Wykonaj jak najwięcej zadań/podpunktów. W zadaniach możesz się odwoływać do aktualnego regulaminu Formula Student. https://www.formulastudent.de/fileadmin/user_upload/all/2027/rules/FS_Rules_2027_v1.0.pdf.

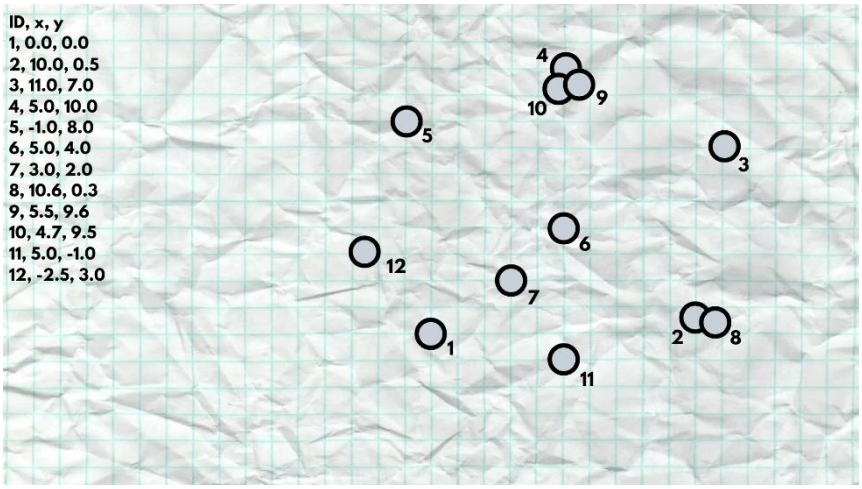
Rozwiązania umieść w repozytorium Github, a link do niego odeślij na maila wiktor.koczkodaj.pwrrt@gmail.com do 16.10.2026r. do godziny 23:59. W tytule maila wpisz [IMIE NAZWISKO] REKRUTACJA AUTONOMY. Pamiętaj o ustawieniu odpowiedniej widoczności repozytorium.

Zadanie 1.

Czujniki systemu percepcji nie zawsze identyfikują położenie pachołków bezbłędnie. Czasem zdarza się, że jeden pachołek przez pomyłkę zostaje zgłoszony kilka razy na różnych, bliskich sobie pozycjach.

Napisz w języku C++ program, który przyjmuje jako wejście plik .txt w następującym formacie:

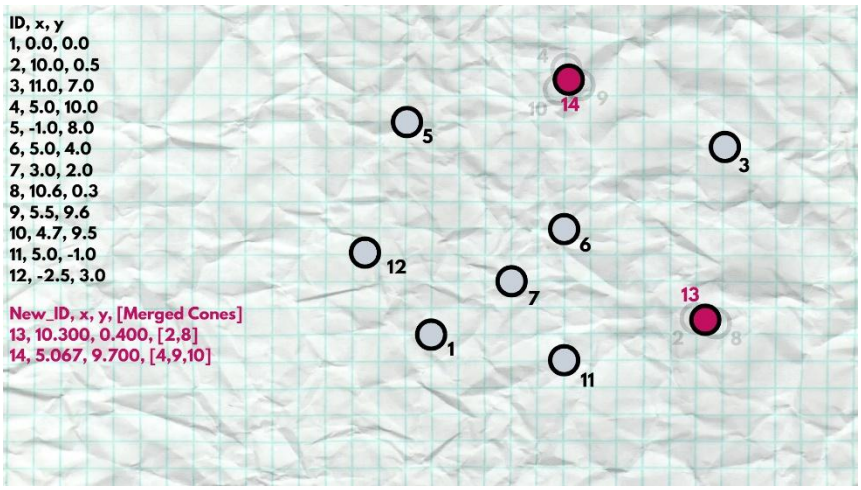
- <n> - pierwsza linia pliku zawiera liczbę punktów w zbiorze
- <m> - druga linia pliku zawiera minimalną dopuszczaną odległość między dwoma pachołkami
- <id> <x> <y> - w kolejnych n liniach znajdować się będzie unikalne ID pachołka, oraz współrzędne x i y oddzielone spacją



Po wczytaniu danych z pliku, twój program powinien:

1.1.

Przyjmując określoną w danych wejściowych minimalną odległość między pachołkami, **przeprowadzić deduplikację**, czyli złączyć ze sobą odczyty znajdujące się wystarczająco blisko siebie, obliczyć pozycję nowego pachołka (uśredniając pozycję składowych), oraz nadać mu unikalne ID. Stare pachołki powinny wówczas zostać usunięte z mapy.

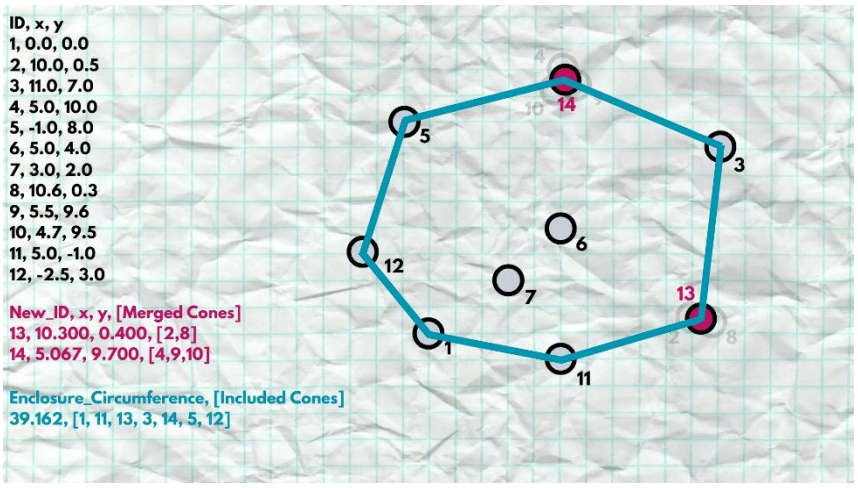


Program powinien zwrócić dane wszystkich nowych pachołków powstałych w trakcie deduplikacji – nadane im ID, ich pozycja, oraz ID pachołków które zostały usunięte z mapy na ich rzecz.

1.2

Znaleźć **najkrótszą otoczkę**. Otoczką nazywamy grupę pachołków, która połączona liniami ogradza wszystkie pachołki, które do niej nie należą. Zadaniem programu jest znaleźć otoczkę o **najkrótszym możliwym obwodzie**.

Jeśli wykonałeś podpunkt 2.1, otoczka powinna być obliczana na podstawie zbioru po deduplikacji (czyt. z nowoutworzonymi pachołkami, bez pachołków usuniętych na ich rzecz – patrz niżej).



Program powinien zwrócić długość znalezionej otoczki oraz listę ID pachołków które ją tworzą. Ich kolejność na liście nie jest istotna.

Przykładowe dane wejściowe:

12

1.0

1 0.0 0.0

2 10.0 0.5

3 11.0 7.0

4 5.0 10.0

5 -1.0 8.0

6 5.0 4.0

7 3.0 2.0

8 10.6 0.3

9 5.5 9.6

10 4.7 9.5

11 5.0 -1.0

12 -2.5 3.0

Przykładowe wyjście (w programie spełniającym oba podpunkty):

CONE 13 10.300 0.400 [2,8]

CONE 14 5.067 9.700 [4,9,10]

ENCLOSURE 39.162 [1,13,11,3,14,5,12]

Kod powinien być udokumentowany, czyli wyjaśnione powinno być zarówno działanie funkcji, jak i ważniejsze fragmenty kodu. Ponadto załącz w repozytorium plik, w którym opiszesz swoje rozwiązanie - sposób działania, zastosowane narzędzia, algorytmy, napotkane edge case'y bądź problemy, oraz wizualizację bądź inny dowód poprawności rozwiązania. Repozytorium powinno zawierać instrukcję uruchamiania programu.

Zadanie 2.

Bolid Formula Student znajduje się 15m przed linią pomiaru czasu. Następnie rusza i w pełni przejeżdża konkurencję Skidpad w sposób zgodny z procedurą ustaloną w regulaminie, zatrzymując się 10m za linią pomiaru czasu. Zapis danych z przejazdu pokazuje jednak, że znajdujące się na aucie czujniki położenia działały błędnie.
W zadaniu nie rozpatrujemy obrotu auta, a jedynie pozycję jego środka ciężkości na dwuwymiarowej mapie widocznej z lotu ptaka.

- Dane z dwóch różnych czujników są zapisane w załączonych plikach *sensor1.csv* oraz *sensor2.csv*, w formacie:
  timestamp,x,y
- *Timestamp* to moment zarejestrowania pomiaru.
- *x* i *y* to absolutne pozycje auta w danym momencie. Dodatnie wartości *x* oznaczają przemieszczenie w przód, a dodatnie wartości *y* oznaczają przemieszczenie w lewo.
- Czujniki operują na różnych częstotliwościach i cechują się różną charakterystyką.

2.1
Przeanalizuj ruch auta raportowany przez oba czujniki przy użyciu wybranego narzędzia do wizualizacji bądź innej formy podglądu. Zdefiniuj dokładniej **charakter błędu** pomiaru obydwu sensorów. Zastanów się, co może spowodować tego typu błędy pomiarowe w rzeczywistości i zapisz swoje podejrzenia co do źródła nieprawidłowości w dokumencie tekstowym.

2.2
Korzystając z analizy przeprowadzonej na rzecz poprzedniego podpunktu oraz wybranych algorytmów, napisz w języku C++ program, który przyjmuje jako wejście dwa załączone pliki .csv, a następnie nasłuchuje w terminalu na wartość liczbową T z zakresu *0.000 – 33.000.*

Jeśli program otrzyma wartość z prawidłowego zakresu, **zwraca pozycję auta w dokładnym momencie T** – czyli np. jeśli T=6, program musi znaleźć lokalizację auta dokładnie w szóstej sekundzie (6.000s) przejazdu w formacie *x y* i wypisać ją w terminalu.

Pamiętaj, że możliwe jest, że żaden z sensorów nie odnotował pomiaru dokładnie w momencie T.

Możesz procesować sygnały (np. dodać mnożniki bądź przesunięcia) w dowolny sposób tak długo, jak będzie on miał analityczne uzasadnienie.

Przykładowe wywołanie:

<< ***Podaj T***

>> *38.163*

<< ***Wartosc spoza zakresu!***

<< ***Podaj T***

>> *26.821*

<< ***7.132 12.982***

Kod powinien być udokumentowany, czyli wyjaśnione powinno być zarówno działanie funkcji, jak i ważniejsze fragmenty kodu. Ponadto załącz w repozytorium plik, w którym opiszesz swoje rozwiązanie - sposób działania, zastosowane narzędzia, algorytmy, napotkane edge case'y bądź problemy, oraz wizualizację bądź inny dowód poprawności rozwiązania. Repozytorium powinno zawierać instrukcję uruchamiania programu.

Znajomość własnych rozwiązań i umiejętność ich wytłumaczenia będzie weryfikowana.